Libraries and Tools

COURSE EXERCISES

Exercise 1 : Inheritance and Class Design 2

Exercise 2 : Command-line Programs 7

Exercise 3 : The Modern Toolbelt 11

Exercise 4 : Scraping a Page Into a Database 15

Exercise 5 : SQLAlchemy 20

Exercise 5 : NumPy 24

Exercise 6 : Pandas 29

APPENDIX

Exercise 9 : Running Other Programs 35

Exercise 1 : Inheritance and Class Design

A fundamentals course ends with an `Employee` class much like the one below. In this exercise you keep that class and grow a family of employees around it, so that a payroll run works the same whether a person is paid by the hour, by the month, or by the hour plus a bonus. Inheritance is the only new idea here; everything else is ordinary class work.

Work in a folder of your own, for example `work/01-inheritance/`. The reference solution is in `solutions/solution01-inheritance/`.

Step 1 : The employee you already have

Create `employee.py` and put a plain `Employee` class in it, with a name, an ID and a `calc_salary` method that returns zero. Nothing here is new :

```
class Employee:
    def __init__(self, name: str, employee_id: int) -> None:
        self.name: str = name
        self.employee_id: int = employee_id

    def calc_salary(self) -> float:
        return 0.0
```

Add a `main` function that builds one employee and prints the name, the ID and the salary, and call it from the usual guard :

```
if __name__ == "__main__":
    main()
```

Run it with `uv run employee.py`. One line of output is the whole goal of this step; it proves the file is where you think it is and that `uv` is working.

Step 2 : An employee paid by the hour

Add a second class in the same file. `WageEmployee` is a kind of `Employee`, so it names `Employee` as its base class, adds an hourly wage and a number of hours, and replaces `calc_salary` with a version that multiplies the two :

```
class WageEmployee(Employee):
    def __init__(self, name: str, employee_id: int, wage: float, hours:
float) -> None:
        super().__init__(name, employee_id)
        self.wage: float = wage
        self.hours: float = hours
```

Write `calc_salary` yourself. Then create a `WageEmployee`, print its `name`, and notice that you never wrote a `name` attribute in this class.

Now delete the `super().__init__(...)` line, run the program again, and read the error carefully. Put the line back. That error is the single most common mistake with inheritance, and it is worth recognising once on purpose.

Step 3 : Two more kinds of employee

Add `ManagerEmployee`, which is also a kind of `Employee` but is paid a fixed amount each month. Its `calc_salary` returns that amount.

Then add `ProgrammerEmployee`. A programmer is paid like a wage employee and gets a bonus on top, so its base class is `WageEmployee` rather than `Employee`. Its `calc_salary` should reuse the version it inherits rather than repeating the multiplication :

```
class ProgrammerEmployee(WageEmployee):
    def calc_salary(self) -> float:
        base = super().calc_salary()
        return base + ...
```

Give it `lines` and `bugs` attributes and decide for yourself what the bonus is worth. Something like one euro a line, minus twenty a bug, is fine.

Print the salary of one of each class.

Step 4 : Printing an employee properly

Put `__str__` and `__repr__` on `Employee` only, and check that all four classes get them :

```
    def __str__(self) -> str:
        return f"{self.name} (#{self.employee_id})"

    def __repr__(self) -> str:
        return f"{type(self).__name__}(name={self.name!r}, id=
{self.employee_id})"
```

Use `type(self).__name__` rather than writing the class name as text, so that a `ManagerEmployee` says so. Print one employee directly, then print a list of employees, and note which of the two methods each case used.

Step 5 : One loop over all of them

Create `company.py` next to `employee.py`, import the classes from it, and build a list holding one of each :

```
from employee import Employee, ManagerEmployee, ProgrammerEmployee,
WageEmployee
```

Write a `payroll` function that takes a `list[Employee]`, loops over it, prints each employee with the salary, and returns the total :

```
def payroll(staff: list[Employee]) -> float:
    ...
```

The body must not mention `WageEmployee`, `ManagerEmployee` or `ProgrammerEmployee` anywhere. Each call to `calc_salary` finds the right version on its own, and that is polymorphism.

Step 6 : Asking what an employee is

Add a function that counts how many of the staff are managers, using `isinstance` rather than comparing `type(...)` with a class. Print the count.

Then check `isinstance(a_programmer, WageEmployee)` and explain to yourself why it is true. Check `issubclass(ProgrammerEmployee, Employee)` as well.

Step 7 : Turning the base class into a contract

Right now nothing stops someone writing a new kind of employee and forgetting `calc_salary` ; the payroll would silently pay them zero. Make that impossible by turning `Employee` into an abstract base class :

```
from abc import ABC, abstractmethod


class Employee(ABC):
    @abstractmethod
    def calc_salary(self) -> float:
        """Every kind of employee has to say how it is paid."""
```

Keep `__init__` , `__str__` and `__repr__` concrete; only `calc_salary` becomes abstract. Then try to create a plain `Employee(...)` and read the `TypeError` . Write a `SalesEmployee` that inherits from `Employee` but supplies no `calc_salary` , try to construct one, and confirm the failure happens at construction rather than at payroll time.

Remove `SalesEmployee` again, or finish it, whichever you prefer.

Step 8 : Optional, a company that has employees

A `Company` is not a kind of employee, so it must not inherit from one. It holds a list of them instead, which is composition :

```
class Company:
    def __init__(self, name: str) -> None:
        self.name: str = name
        self.staff: list[Employee] = []

    def hire(self, employee: Employee) -> None:
        ...
```

Give it `hire`, `fire` and a `payroll` method that reuses the function from Step 5. Then add a `breakdown` method that reports the total per class, using `type(employee).__name__` as the key of a dictionary.

Say both relationships out loud as a check on the design : a programmer *is a* wage employee, and a company *has* employees.

Exercise 2 : Command-line Programs

In this exercise you take a script that reads one hard-coded log file and turn it into a small command-line tool called `logtool`, with two subcommands, generated help, and an exit status the shell can act on.

Step 1 : Take an argument

Create a directory `exercise2` and a Python module `logtool.py` in it, and copy `data/cli-sample.log` into the same directory so that everything you need is in one place. Start from the script below, which is also shipped as `solutions/solution02-command-line-programs/logreport_start.py`, where the only difference is that it looks for the log in the course's `data/` directory rather than next to itself. It counts how many lines of the sample log carry each level, and the filename is written into the source :

```
from collections import Counter
from pathlib import Path

LOGFILE = Path(__file__).parent / "cli-sample.log"


def count_levels(path: Path) -> Counter[str]:
    return Counter(line.split()[2] for line in
path.read_text(encoding="utf-8").splitlines())


def main() -> None:
    for level, count in count_levels(LOGFILE).most_common():
        print(f"{level:<6}{count}")


if __name__ == "__main__":
    main()
```

Run it once so that you know what its output looks like. Then give it an `ArgumentParser` with a single positional argument for the file to read, converted to a `Path` :

```
import argparse

parser = argparse.ArgumentParser(description="Summarise a log file.")
parser.add_argument("logfile", type=Path, help="the log file to read")
args = parser.parse_args()
```

Use `args.logfile` instead of the `LOGFILE` constant, and run the script again, this time passing `cli-sample.log` on the command line. Then run it with `--help` and confirm that the help text you never wrote is there.

Step 2 : Two subcommands

The tool is going to do two jobs, so it needs two verbs : `count` , which is the report you already have, and `errors` , which prints the lines at one level.

Move the body of the current program into a function `cmd_count` , and write a second function `cmd_errors` next to it. Both take the parsed arguments and return an integer :

```
def cmd_count(args: argparse.Namespace) -> int:
    ...
    return 0


def cmd_errors(args: argparse.Namespace) -> int:
    matches = [line for line in read_lines(args.logfile) if f"
{args.level} " in line]
    ...
```

Now register the two verbs with `add_subparsers` , and attach each function to its verb with `set_defaults` , so that no `if` statement over the verb name is needed :

```
verbs = parser.add_subparsers(dest="command")

counting = verbs.add_parser("count", help="count the lines per level")
counting.set_defaults(handler=cmd_count)
```

Give the `errors` verb a `--level` option with a default of `ERROR`, so that `logtool errors --level WARN` works too. Both verbs need the `logfile` argument. You can add it to each subparser separately, or declare it once on a parent parser and pass `parents=[common]` to both, which is what the solution does.

Calling the handler is then a single line :

```
return args.handler(args)
```

Check that `logtool.py count`, `logtool.py errors` and `logtool.py errors --level WARN` all work, and that `logtool.py count --help` shows the help for that verb only.

Step 3 : An exit status that means something

Give the program the standard entry point, so that the value it returns becomes the exit status of the process :

```
def main(argv: list[str] | None = None) -> int:
    ...


if __name__ == "__main__":
    raise SystemExit(main())
```

Use three statuses, following the usual convention :

- `0` when the tool did its job and there is nothing to report
- `1` from `errors` when it actually found lines at the requested level
- `2` when the file cannot be read at all

For the last one, wrap the call to the handler in a `try` block, catch `OSError`, and print the reason to **stderr** rather than stdout, because a message is not data :

```
except OSError as error:
    print(f"logtool: {error.strerror}: {args.logfile}",
file=sys.stderr)
    return 2
```

Now test it the way the shell would, running each command and asking for the status afterwards :

```
uv run logtool.py errors cli-sample.log ; echo $?                # 1,
there are ERROR lines
uv run logtool.py errors cli-sample.log --level FATAL ; echo $?  # 0,
there are none
uv run logtool.py count /no/such/file ; echo $?                  # 2,
message on stderr
```

Finally, run the tool with no arguments at all. Decide what should happen, and make it happen deliberately : either print the help and return `0`, or let `add_subparsers(required=True)` refuse the call with status `2`.

Step 4 : Optional, make it a standalone command

Turn the file into a program that needs nothing but `uv` to run. Add a shebang and a PEP 723 metadata block as the very first lines, above the docstring :

```
#!/usr/bin/env -S uv run --script
# /// script
# requires-python = ">=3.12"
# dependencies = []
# ///
```

On Linux or macOS, `chmod +x logtool.py` and then run `./logtool.py count` directly. Then add `rich` to the `dependencies` list and use a `rich.table.Table` for the output of `count`, as demo 12 does. The dependency is installed the first time the script runs and cached afterwards, and the file is still one file.

Exercise 3 : The Modern Toolbelt

In this exercise you take a plain reporting script and give it the two things that make a command-line tool feel finished : a progress bar while it works, and a coloured table when it is done. The whole exercise is about thirty lines of code, and every step runs on its own.

Step 1 : Prove the libraries are there

Make a file `health.py` and check that the three libraries import and print something. Do this before anything else, so that nothing later fails for a reason that has nothing to do with your code :

```
from rich.console import Console

console = Console()
console.print("[bold green]toolbelt ready[/]")
```

Run it with `uv run health.py`. If you see the words in green, the environment is correct and the rest of the exercise will work.

Step 2 : Read the inventory

Read `data/servers.csv` into a `list[dict[str, str]]` with `csv.DictReader`. Resolve the path from the file rather than from the working directory, so that the script runs from anywhere :

```
from pathlib import Path

DATA = Path(__file__).resolve().parent / "data"


def load() -> list[dict[str, str]]:
    ...
```

Print how many rows you read. You should get fourteen.

Step 3 : Add a progress bar

Write a `check(server: dict[str, str]) -> str` function that decides whether a host is `"ok"`, `"warning"` or `"full"`, based on `used_gb` divided by `disk_gb`. Call `time.sleep(0.2)` inside it, so that the bar has something to show.

Now loop over the servers and wrap the loop in `tqdm`. You do not change the loop body at all :

```
from tqdm import tqdm

for server in tqdm(servers, desc="Checking", unit="host"):
    ...
```

One host in the file has no `used_gb` reading. Decide what your `check` should return for it rather than letting it raise.

Step 4 : Report anything alarming, without breaking the bar

While the bar is running, print a line for every host that is not `"ok"`. Use `tqdm.write` rather than `print`, and see what happens if you use `print` instead :

```
tqdm.write(f"{server['hostname']}: {fraction:.0%} used")
```

Step 5 : Print a table at the end

Build a `rich.table.Table` with a column for the host, the site, the disk size and the usage, and add one row per server. Right-align the numbers, and colour the usage cell by how alarming it is :

```
from rich.table import Table

table = Table(title="Disk usage", header_style="bold magenta")
table.add_column("Host", style="cyan")
table.add_column("Used", justify="right")
```

Remember that the colour goes in the cell text as markup, `f"[red]{value}[/]"`, not as an argument to `add_row`.

Step 6 : Sort and summarise

Sort the rows so the fullest disks are at the top, since that is the only part of the report anybody reads. Then add a footer row giving the total disk space and the overall percentage used, with `table.show_footer = True` and `table.columns[0].footer = ...`.

Step 7 *Optional* : Turn it into a command line tool

Give the script a `--site` option so it reports on one site only, and a `--threshold` option that sets the percentage at which a host counts as alarming. Do it with `typer`, and note how much of the argparse version from module 2 you no longer have to write :

```
import typer
from typing import Annotated

app = typer.Typer(add_completion=False)


@app.command()
def report(
    site: Annotated[str | None, typer.Option("--site", "-s")] = None,
    threshold: Annotated[int, typer.Option("--threshold", "-t")] = 85,
) -> None:
    """Report disk usage across the estate."""
```

Run it with `--help` and read the help page you did not write.

Step 8 *Optional* : Swap tqdm for rich

Replace the `tqdm` bar with `rich.progress.track`, so that the bar and the table are drawn by the same library and share a colour scheme. Note that inside a rich progress block an ordinary `print` no longer corrupts the display.

Exercise 4 : Scraping a Page Into a Database

In this exercise you build the three steps of a scraper against a real site : you fetch a page from `books.toscrape.com` , parse the twenty books out of the markup, write them into a SQLite database with parameterised inserts, and query the result back. The site exists to be scraped, so you are welcome there as long as you are polite about it.

Work in a file of your own, for example `scrape_books.py` , and run it with `uv run scrape_books.py` from the repository root. The reference answer is in `solutions/solution04-web-scraping/` .

Step 1 : Prove the fetch and the parser work

Fetch the page and make a soup from it. Always pass a `timeout` , because `requests` has no default and a server that goes quiet would hang your script for ever :

```
import requests
from bs4 import BeautifulSoup

URL =
"https://books.toscrape.com/catalogue/category/books/mystery_3/index.ht
ml"
html = requests.get(URL, timeout=10).text
soup = BeautifulSoup(html, "lxml")
```

Print two things : the page title, and how many `article` elements with class `product_pod` the page contains. You should see the word `Mystery` and the number `20` . That is the whole of step one.

If the title comes out full of newlines, tidy it with `" ".join(soup.title.text.split())` .

Step 2 : One book

Take the first `article.product_pod` and pull three fields out of it :

- the title, which is the `title` attribute of the link inside the `h3`
- the price, which is the text of the `p` with class `price_color`
- the rating, which is the second class name on the `p` with class `star-rating`

```
pod = soup.select_one("article.product_pod")
link = pod.select_one("h3 a")
print(link["title"], link["href"])
print(pod.select_one("p.price_color").text)
print(pod.select_one("p.star-rating")["class"])
```

Note that the price text carries a currency sign and the rating is a word rather than a number. Deal with both : strip everything that is not a digit or a dot out of the price and turn it into a `float`, and map the words `One` to `Five` onto the numbers 1 to 5 with a dictionary.

Step 3 : All twenty, as records

Write a function that does step 2 for every book on the page and returns a list of dictionaries, one per book :

```
def parse_books(html: str, page_url: str) -> list[dict]:
    ...
```

Each record should carry `title`, `price_gbp` as a `float`, `rating` as an `int`, `availability` as text, and `url`. Build the url with `urllib.parse.urljoin` against the page address so the stored link is absolute :

```
url = urljoin(page_url, link["href"])
```

Print the first three records and the number of records. After this function returns, no part of your program should ever mention HTML again.

Step 4 : Into the database

Create `data/exercise-books.db` if it is not there, with one table. Give `url` a `UNIQUE` constraint, because one page is one book :

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS books (
    id        INTEGER PRIMARY KEY,
    title     TEXT    NOT NULL,
    price_gbp REAL    NOT NULL,
    rating    INTEGER NOT NULL,
    url       TEXT    NOT NULL UNIQUE,
    last_seen TEXT    NOT NULL
);
"""
```

Insert the records with `executemany` and placeholders, never with a formatted string. Named placeholders take your dictionaries directly, once you have added the timestamp :

```
INSERT = "INSERT INTO books (title, price_gbp, rating, url, last_seen)
VALUES (...)"
seen_at = datetime.now(UTC).isoformat(timespec="seconds")
conn.executemany(INSERT, [book | {"seen_at": seen_at} for book in
books])
conn.commit()
```

Remember that `sqlite3` does not commit for you. Print how many rows the table holds when you are done.

Step 5 : Query it back

With the data in a table, the questions are ordinary SQL. Answer three of them and print the results :

- the five most expensive books, title and price
- how many books there are at each rating, and the average price per rating
- the number of books priced under twenty

```
rows = conn.execute(
    "SELECT rating, COUNT(*), ROUND(AVG(price_gbp), 2) FROM books GROUP
BY rating"
).fetchall()
```

Setting `conn.row_factory = sqlite3.Row` before the query lets you write `row["rating"]` instead of `row[0]`, which is worth it as soon as a query has more than two columns.

Step 6 : Run it twice

Run your script a second time. If it fails with `sqlite3.IntegrityError` on the unique url, that is the constraint doing its job.

Make the script safe to re-run by turning the insert into an upsert :

```
INSERT INTO books (...) VALUES (...)
ON CONFLICT(url) DO UPDATE SET
    price_gbp = excluded.price_gbp,
    last_seen = excluded.last_seen
```

Run it twice more and confirm the row count stays at twenty while `last_seen` moves.

Step 7 : Optional, the second page, politely

The Mystery category has a second page of twelve more books :

```
PAGE_URLS = [

"https://books.toscrape.com/catalogue/category/books/mystery_3/index.ht
ml",

"https://books.toscrape.com/catalogue/category/books/mystery_3/page-
2.html",
]
```

Loop over both, parse each, and upsert the lot. Three things make the difference between a scraper a site tolerates and one it blocks :

```
session = requests.Session()
session.headers["User-Agent"] = "MyScraper/1.0 (you@example.com)"
time.sleep(2)          # between requests, not before the first one
```

Use one `Session` so the connection is reused, name your program in the `User-Agent`, and sleep between the two requests. The row count should go from twenty to thirty two, and running the whole thing again should leave it at thirty two.

Exercise 5 : SQLAlchemy

In this exercise you point SQLAlchemy at the database the scraping module filled in, describe its table as a Python class, and then build a small catalogue of your own with a relationship in it. Everything you write goes into `data/exercise-library.db`, so the scraped `data/books.db` is only ever read and cannot be damaged.

Step 1 : Connect to something that already exists

Make a file `library.py`. Create an engine pointing at the scraped database and ask it what is in there. This step exists to prove the environment works :

```
from pathlib import Path
from sqlalchemy import MetaData, create_engine

DATA = Path(__file__).resolve().parent / "data"
engine = create_engine(f"sqlite:///{DATA / 'books.db'}")

metadata = MetaData()
metadata.reflect(bind=engine)
print(list(metadata.tables))
```

You should see `['books']`. Now print the column names of that table, and then the number of rows in it. If `data/books.db` does not exist, run `uv run demos/04-web-scraping/12_rebuilding_from_seed.py` first, which builds it from the checked-in seed without touching the network.

Step 2 : Describe the table as a class

Reflection is convenient, but it gives you tuples. Write a `Book` model that matches the table you just inspected, so that queries hand back objects instead :

```
class Base(DeclarativeBase):
    pass


class Book(Base):
    __tablename__ = "books"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    # add price_gbp, rating and category
```

Give it a `__repr__` so that printing one is readable. Then open a `Session` and print the first three books.

You do not need `create_all()` here; the table already exists and your class is describing it, not creating it.

Step 3 : Ask it questions

Using `select()` and the session, answer each of these. Print the answer with a line saying what it is :

- Every book rated 5, ordered by price, most expensive first.
- The number of books costing more than £30.
- The average price at each rating, as a small table.

For the last one you want `func.count()` and `func.avg()`, and `group_by`. Compare what you write with the SQL you would have written by hand; they should look similar enough to be reassuring.

Step 4 : Your own database, with two tables

Now build something of your own in `data/exercise-library.db`. Two models :

- `Shelf`, with an `id` and a `name`.
- `Volume`, with an `id`, a `title`, and a foreign key to `Shelf`.

Wire them together with a relationship in both directions :

```
class Shelf(Base):
    __tablename__ = "shelf"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(60))
    volumes: Mapped[list[Volume]] =
relationship(back_populates="shelf")
```

Call `Base.metadata.create_all(engine)` against the new database, then create two shelves with a few volumes each. Add only the shelves to the session, not the volumes, and check that the volumes are saved anyway.

Step 5 : Read it back both ways

Print every shelf with the titles on it, using the relationship attribute. Then print the same information again as a single query with an explicit `join`.

Both should produce the same lines. Satisfy yourself about which one issued how many queries; passing `echo=True` to `create_engine` will show you every statement as it is sent, which is the quickest way to see the difference.

Step 6 : Change something

Move one volume from one shelf to the other by assigning to its `shelf` attribute, then commit. Print both shelves again to confirm it moved.

Notice that you did not write an `UPDATE` statement, and that you did not touch the foreign key column yourself.

Step 7 : Optional, hand it to pandas

Load your `volume` table into a DataFrame with `pandas.read_sql`, passing the engine rather than a connection. Group it by shelf and count the rows.

Then do the same aggregation in SQL instead, and compare. Which one you would reach for in real work depends on how big the table is and on what you want to do next, and it is worth having an opinion about both.

Exercise 5 : NumPy

In this exercise you take a day of CPU readings off five servers and answer questions about them with arrays instead of loops. The data is already in the repository as `data/server-load.csv` : twenty-four hourly readings for `web01` , `web02` , `web03` , `db01` and `cache01` , as a percentage. By the end you will have found the busiest server, the busiest hour, and the one host that misbehaves in the middle of the night.

Work in a folder `numpy_load/` , one file per step, and keep the files as you go. Every answer in this exercise is one or two lines. If you find yourself writing a `for` loop over an array, stop and look for the expression instead.

Step 1 : Load the file

Create `01_load.py` .

Read the CSV into an array. The first row is a header and the first column is the hour, so neither is a number you want in the array :

```
from pathlib import Path

import numpy as np

DATA = Path(__file__).resolve().parents[2] / "data"

load = np.loadtxt(DATA / "server-load.csv", delimiter=",", skiprows=1,
usecols=(1, 2, 3, 4, 5))
```

Print `load.shape` , `load.dtype` and `load.size` . You should see `(24, 5)` : twenty-four rows of hours, five columns of servers. Print the first two rows as well, so you can see real numbers on the screen.

That is the whole of step one. If it printed `(24, 5)` , your environment works and the rest of the exercise will run.

Step 2 : Summarise each server

Create `02_per_server.py` and load the array again.

Each column is one server, so a per-server answer means collapsing the hours. That is `axis=0` : the rows are the dimension that disappears.

Compute the mean, the minimum and the maximum for each of the five servers, and print them next to their names :

```
SERVERS = ["web01", "web02", "web03", "db01", "cache01"]

means = load.mean(axis=0)
for name, mean in zip(SERVERS, means, strict=True):
    ...
```

Then find the busiest server. `np.argmax` gives you a position rather than a value, and `SERVERS[position]` turns that position back into a name.

Check your answer against the shapes : `load.mean(axis=0)` must have shape `(5,)` . If you got `(24,)` , you used the other axis.

Step 3 : Summarise each hour

Create `03_per_hour.py` .

Now collapse the other way. A per-hour answer keeps the rows and loses the servers, which is `axis=1` .

Compute the mean load per hour, then print :

1. The busiest hour of the day, and its mean.
2. The quietest hour, and its mean.
3. The total load per hour, so you can see the shape of the working day.

The hour number is the row number, because the rows are in order from 00:00, so `np.argmax` gives you the hour directly.

> *If you are not sure which axis you want, print the shape of the result. One number per hour means twenty-four numbers.*

Step 4 : Find the readings that matter

Create `04_alerts.py` .

Build a boolean mask of every reading above 80 percent :

```
busy = load > 80.0
```

Print how many readings are over the line, using `busy.sum()` . Then use `np.nonzero(busy)` to get the positions. It returns one array per dimension, so unpack it into two names :

```
hours, servers = np.nonzero(busy)
```

Loop over those two arrays together and print one line per alert, naming the server and the hour, for example `db01 at 02:00 -> 93.3 %` . This is the one loop in the exercise you are allowed to write, because you are formatting text rather than computing numbers.

Then answer the same question a second way : which servers ever go over 80 percent at all? `busy.any(axis=0)` gives you one `True` or `False` per server, which you can use to index the server names once you have made them an array.

Step 5 : Compare each server to its own baseline

Create `05_baseline.py` .

A raw percentage is not very interesting; the interesting question is how far each reading is from normal *for that server*. `cache01` sitting at 10 percent is fine, `cache01` at 40 percent would not be.

Take the median of each server, then subtract it from that server's column. The medians come out as shape `(5,)` and the array is `(24, 5)`, so they line up from the right and broadcast across every row with no reshaping needed :

```
baseline = np.median(load, axis=0)
deviation = load - baseline
```

Print the shape of `baseline`, and the largest deviation for each server. Then find the single worst deviation in the whole table. `np.argmax` on a two-dimensional array gives you a position into the flattened array, so use `np.unravel_index` to turn it back into a row and a column :

```
hour, server = np.unravel_index(np.argmax(deviation), deviation.shape)
```

Finally, make a copy of `load` and clip every reading in the copy to a ceiling of 90 percent, without disturbing the original :

```
capped = load.copy()
capped[capped > 90.0] = 90.0
```

Print the maximum of both arrays to prove the original survived. Leave the `.copy()` out and run it again to see what happens when you forget.

Optional : band every reading

Create `06_bands.py` if you have time.

Use `np.where` to turn the whole table into labels in one expression : `critical` above 90, `warning` above 60, and `fine` otherwise. Nesting one `np.where` inside another gives you the three bands.

Then count how many readings fall into each band. Comparing the label array against a string gives you a mask, and a mask sums to a count :

```
labels = np.where(load > 90.0, "critical", np.where(load > 60.0,
"warning", "fine"))
print("critical :", (labels == "critical").sum())
```

For a last flourish, print a per-server count of warnings by summing that mask along `axis=0`, and write the hourly means out to a new CSV in `data/` with `np.savetxt`.

Exercise 6 : Pandas

In this exercise you take the book catalogue that the scraping module put into SQLite, pull it into pandas, and turn it into the spreadsheet somebody upstairs actually asked for. You will read a database with `read_sql`, filter it, add a column, summarise it with `groupby`, and write the answer out as an Excel workbook with two sheets. Nothing here is longer than a couple of lines; the work is choosing the right couple of lines.

Work in a folder `book_report/`, one file per step, and keep the files as you go. Each step reloads the data at the top, which costs nothing on twenty rows and means any file runs on its own.

> ***If*** ***`data/books.db`*** ***is not there yet****, the scraping module has not been run on this machine. Build it with* `uv run demos/04-web-scraping/12_rebuilding_from_seed.py`*, which recreates the table from the checked-in* `data/books-seed.sql` *without scraping anything.*

Step 1 : Open the database

Create `01_load.py`.

Connect to the database, ask it what tables it has, and read one of them into a DataFrame :

```
import sqlite3
from pathlib import Path

import pandas as pd

DATA = Path(__file__).resolve().parents[2] / "data"

conn = sqlite3.connect(DATA / "books.db")
print(pd.read_sql("SELECT name FROM sqlite_master WHERE type =
'table'", conn))
books = pd.read_sql("SELECT * FROM books", conn)
conn.close()

print(books.shape)
print(books.head())
```

You should see one table called `books`, a shape of `(20, 9)`, and five rows of book titles.

That is the whole of step one. If those twenty rows printed, your environment works and the rest of the exercise will run.

Step 2 : Look at what you loaded

Create `02_inspect.py` and read the table again.

Print `books.info()` and `books.describe()`. Between them they answer the three questions worth asking about a file you did not write yourself : how many rows are there, what type is each column, and is anything missing.

Then answer two more, using what the demos showed :

1. How many missing values are there in each column? `isna()` gives a mask and a mask sums to a count.
2. What distinct values does the `category` column hold, and how many books are in each? `value_counts()` does both at once.

Note the dtype of `price_gbp` and the dtype of `rating`. One is a float and one is an integer, and pandas worked that out from the database rather than being told.

Step 3 : Filter down to the rows that matter

Create `03_filter.py`.

Build the filters as named variables first, then combine them. A mask with a name is a mask you can read six weeks later :

```
well_rated = books["rating"] >= 4
in_stock = books["availability"] == "In stock"
affordable = books["price_gbp"] < 40.0
```

Print how many rows each single filter matches, then print the rows that satisfy all three. Remember `&` rather than `and`, and remember the parentheses around each comparison when you write the combination in one line.

Then do the same thing two other ways, so you have seen them :

1. `books[books["rating"].isin([4, 5])]`, which replaces a chain of ORs.
2. `books[books["price_gbp"].between(20.0, 40.0)]`, which replaces two comparisons and includes both ends.

Step 4 : Add the columns the report needs

Create `04_derive.py`.

The catalogue is priced in pounds and the report wants euros. Add a column, rounded to two decimals :

```
books["price_eur"] = (books["price_gbp"] * RATE).round(2)
```

Then add a `band` column that says `budget` under 20 euros, `mid` under 45, and `premium` above that. `np.where` nested inside another `np.where` does it in one expression, or use `pd.cut` if you would rather name the edges.

Finally sort the whole thing by price, most expensive first, and print the top five titles with their two prices. `sort_values("price_eur", ascending=False).head(5)` is the whole of it, and `nlargest(5, "price_eur")` says the same thing in fewer words.

Step 5 : Summarise by rating

Create `05_group.py`.

This is the `GROUP BY` of the exercise. Produce one row per star rating, with four columns : how many books have that rating, the mean price, the cheapest and the dearest.

Use named aggregation so the output columns arrive with the names you want rather than names you have to fix afterwards :

```
per_rating = (
    books.groupby("rating")
    .agg(
        titles=("title", "count"),
        mean_price=("price_eur", "mean"),
        ...
    )
    .round(2)
    .reset_index()
)
```

Print it. Then answer the question it raises : does a higher rating cost more? Sorting the result by `mean_price` is enough to see.

Step 6 : Export to Excel

Create `06_export.py`.

Build both frames again : the filtered list of well-rated books from step 3 with the derived columns from step 4, and the per-rating summary from step 5. Then put them in one workbook, one sheet each :

```
with pd.ExcelWriter(DATA / "book-report.xlsx", engine="openpyxl") as
workbook:
    shortlist.to_excel(workbook, sheet_name="shortlist", index=False)
    per_rating.to_excel(workbook, sheet_name="by rating", index=False)
```

Pass `index=False` on both, because the row numbers are pandas' business and not the reader's. Write a CSV of the shortlist as well with `to_csv`, for whoever does not have Excel.

Print the path and the sheet names at the end, then open the file and check that it is really a spreadsheet.

Optional : a chart and a second table

Create `07_chart.py` if you have time.

Two things, either order.

A picture. Plot the mean price per rating as a bar chart. Select the non-interactive backend before importing `pyplot`, so the script writes a file instead of waiting for you to close a window :

```
import matplotlib

matplotlib.use("Agg")
```

Then `per_rating.plot(kind="bar", x="rating", y="mean_price")`, take `.get_figure()` from the axes it returns, and `savefig` it into `data/`.

A join. Build a small lookup frame giving each star rating a word :

```
words = pd.DataFrame({"rating": [1, 2, 3, 4, 5], "verdict": ["poor",
...]})
```

`merge` it onto the summary on `rating` and print the result, so the report reads in English rather than in integers. Then try the same merge with `how="left"` after deleting one row from the lookup, and use `indicator=True` to see exactly which rows failed to match. That check is worth doing on every join you ever write.

Exercise 9 : Running Other Programs

APPENDIX

A small appendix exercise, and the one most worth doing on your own Linux machine rather than in the classroom, because there the commands are real and the shell is a POSIX one. You will build a short report about a directory of files by asking other programs for the answers, and then replace half of it with Python that needs no other program at all.

Step 1 : Ask one program one question

Make a file `report.py`. Run `git --version` and print the version it reports, without the surrounding text :

```
import subprocess

finished = subprocess.run(
    ["git", "--version"], capture_output=True, text=True, check=True
)
print(finished.stdout.strip())
```

Change `check=True` to `check=False` and run it against a command that does not exist, such as `["git", "nonsense"]`. Look at `returncode` and at `stderr`.

Step 2 : Read an exit code that means something

Run `grep -c ERROR` against `data/cli-sample.log` and print the count.

Then run it against a pattern that matches nothing. You will get a return code of 1 and an empty count, and that is not a failure; it is `grep` telling you it found nothing.

Write a function that returns the number of matching lines, treating a return code of 0 or 1 as success and anything else as a real error :

```
def count_matches(pattern: str, path: Path) -> int:
    found = subprocess.run(...)
    if found.returncode > 1:
        raise RuntimeError(f"grep failed: {found.stderr.strip()}")
    ...
```

Step 3 : Make it fail safely

Add a check at the top of the script that `grep` is actually installed, using `shutil.which`. If it is missing, print a clear message to `stderr` and exit with a non-zero status, using what you built in block 02.

Test it by asking for a program you know is not there.

Step 4 : Two commands, no shell

Produce a count of each log level in `data/cli-sample.log`, the way you would with `grep -o -E 'INFO|WARN|ERROR' file | sort | uniq -c`, but as separate `subprocess.run` calls passing text between them with `input=`.

Print the result as a small table.

Step 5 : Give it a timeout

Add `timeout=` to every call. Pick something generous, five seconds say, so it never fires in normal use.

Then prove it works by running a command that sleeps for longer than that, and catching `TimeoutExpired`. On Linux, `["sleep", "30"]` will do.

Step 6 : Now throw half of it away

Rewrite Step 4 in pure Python: read the file with `Path.read_text`, and count the levels with a dictionary or `collections.Counter`. No `subprocess` at all.

Compare the two versions. Time them if you like; the difference will be larger than you expect, because the first version starts three processes and the second starts none.

Which one you would rather maintain is the real question, and the answer is not the same for every script.

Step 7 : Optional, something genuinely useful

Point the script at a directory of your own and have it report something you actually want to know, using a program that has no Python equivalent worth writing. Some possibilities :

- `git log --format=%an` piped into a count, for who has touched a repository.
- `systemctl is-active` for a list of services, reporting only the ones that are not running.
- `df -P` parsed into a warning for any filesystem over a threshold you choose.

Keep the rule from the last slide in mind : let the other program do what only it can do, and do the rest in Python.